IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Demostrador y documentación

Cómo entregar un proyecto de IA para que **otra persona pueda ejecutarlo**

Sprint 16 · S16_01 · cuaderno: 45 min en clase (se ejecuta y se lee; ejercicios 1 a 3, en casa)

LA IDEA · QUÉ SE ENTREGA Y CUÁNDO

Entregar es que otra persona lo ejecute

Un modelo que solo vive en tu portátil no es un producto.

Cuándo	Qué tiene que estar hecho
Viernes 4/12	**Demostrador funcionando** (aunque sea básico) y esqueleto de la memoria
Lunes 7 y martes 8/12	**Sin clase**: buen momento para avanzar la memoria en equipo
Miércoles 9/12	Sesgos mitigados, demostrador y model card casi listos; memoria con todos los apartados
Jueves 10/12, 14:30	**Congelación** (desde ahí no se cambia nada): último push. Lo que no esté subido, no cuenta

Se sube a **sprint-16/**

9 ficheros del equipo y **1 hoja por persona**: README, evaluación, demostrador, riesgos, model card y memoria.

Aparte: vídeo de reserva y coevaluación (al docente).

Palabras nuevas

Demostrador: app o cuaderno donde otra persona prueba el modelo.

Model card: la ficha del modelo (qué hace, con qué datos, cuándo no usarlo).

LA IDEA · EL DEMOSTRADOR (CUADERNO, SECCIÓN 1)

El demostrador no reentrena

1 · Se entrena una vez
En el cuaderno o en un script de entrenamiento

→

2 · Se guarda
El modelo con **joblib** y un archivo de **metadatos.json**

→

3 · Se carga
cargar_modelo() lee los dos archivos

→

4 · Se usa
La app, los widgets o la API piden una predicción

LA CARPETA

```
demostrador/
├── modelo/              # joblib, json
├── logica.py            # validar + predecir
├── app.py               # interfaz Streamlit
├── api.py               # (opcional) FastAPI
├── tests/               # pruebas
├── requirements.txt     # versiones fijas
└── README_demostrador.md
```

METADATOS.JSON (EXTRACTO REAL)

```
{
  "campos": {"noches": {"tipo": "entero", "min": 1, "max": 30, ...}, ...},
  "auc_test": 0.746,
  "n_entrenamiento": 4800,
  "semilla": 42,
  "versiones": {"scikit-learn": "1.9.1", ...},
  "limitaciones": "Entrenado con datos sintéticos..."
}
```

LA IDEA · LA LÓGICA SEPARADA DE LA INTERFAZ (CUADERNO, SECCIÓN 2)

La lógica, separada de la interfaz

INTERFACES: CAPAS FINAS

- **app.py** Streamlit
- **widgets** en el cuaderno
- **api.py** FastAPI, opcional
- **tests/** pytest

→

logica.py

```
cargar_modelo()
validar_reserva()
predecir_reserva()
```

Sin nada de interfaz

→

modelo/

```
modelo_cancelaciones.joblib
metadatos.json
```

1 · Errores en español y accionables

Dicen **qué campo falla**, qué se esperaba y qué se escribió. Salida real del cuaderno:

'noches' debe estar entre 1 y 30 (has escrito -3).

2 · Aviso de incertidumbre en cada respuesta

Viaja **con** el resultado: incluye la nota del modelo (AUC de 0,746 en datos de prueba), que se equivoca con frecuencia, y sus límites.

3 · Devuelve datos, no imprime

Un diccionario que cada interfaz muestra como quiera:

probabilidad 0,812 · **nivel** alto · **tasa_base** 0,459 · **aviso** …

Si hay errores, solo devuelve **errores**.

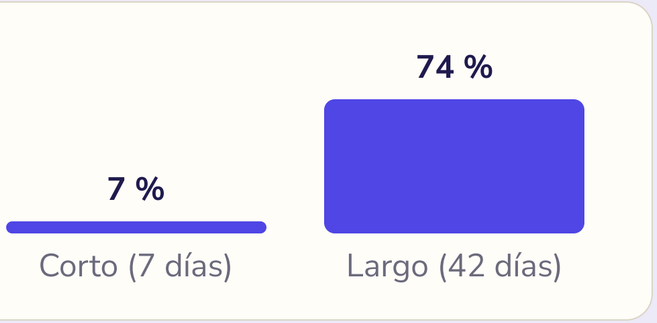
LA IDEA · PRUEBAS DE SENTIDO COMÚN (CUADERNO, SECCIÓN 3)

Un modelo puede fallar sin dar error

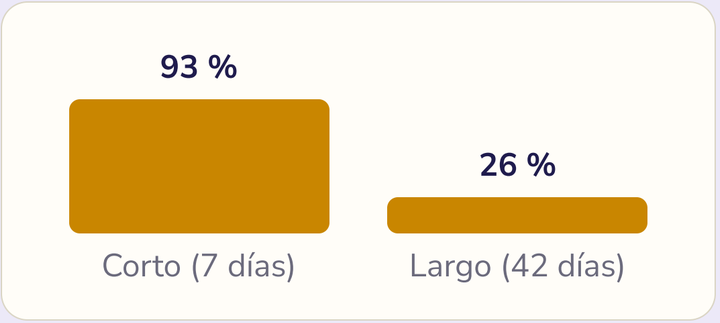
EJEMPLO INVENTADO: ¿SE DEVOLVERÁ TARDE UN PRÉSTAMO?

```
def test_sentido_comun():
    largo = predecir_prestamo({**base, "dias": 42, "novedad": 1})
    corto = predecir_prestamo({**base, "dias": 7, "novedad": 0})
    assert largo["probabilidad"] > corto["probabilidad"]
```

Modelo bien entrenado: **pytest -q** muestra **.**



Etiquetas al revés: **pytest -q** muestra **F**



Además, a mano: cinco entradas

3 normales: resultado coherente.

1 imposible (por ejemplo, 250 años): mensaje claro, sin romperse.

1 vacía: mensaje claro.

LA IDEA · OPCIONES DE DEMOSTRADOR (CUADERNO, SECCIONES 4 A 6)

Cuatro formas de enseñar el modelo

Opción	Cuándo	Ventaja	Inconveniente	En el cuaderno
Streamlit (recomendada)	Modelos tabulares, series, texto	Web interactiva con pocas líneas	Hay que instalarlo (**streamlit run**)	Sección 4
Gradio	Imágenes, audio, chat	Muy rápido para subir una foto	Menos control del diseño	Sin sección
Cuaderno con widgets	Plan B, sin instalar nada	Funciona en Colab	Menos «producto»	Sección 5
API con FastAPI (opcional)	Si otro sistema lo consulta	Estándar en producción	Necesita un cliente que la use	Sección 6

app.py es una capa fina

1. Carga el modelo una vez (**cargar_modelo**, en caché).
2. Dibuja el formulario con **meta["campos"]**.
3. Llama a **predecir_reserva**.
4. Muestra el resultado o los errores.

En Colab, Streamlit no se abre

Hace falta un túnel. Para el plan B usad la opción de **widgets**, que funciona dentro del cuaderno.

En clase solo se revisa que **app.py** compila; se lanza desde una terminal.

EJERCICIOS 1 A 3 Y MINI-RETO · CUADERNO (CASA)

Para casa: tres ejercicios y un mini-reto

Ejercicio 1 · **validar_coherencia**

Se pide: una función que devuelva una lista de errores (vacía si todo va bien) con dos reglas que cruzan campos.

Idea: la validación campo a campo no ve las incoherencias entre dos campos.

Ejemplo propio: en la biblioteca, un préstamo de 3 días no puede llevar renovaciones.

Ejercicio 2 · **mensaje_errores**

Se pide: convertir la lista de errores en un único texto para la persona que usa el demostrador.

Idea: una línea por error, unidas con saltos de línea (**join**).

Ejemplo propio: **["falta carnet", "días fuera de rango"]** pasa a un texto con cabecera y dos líneas.

Ejercicio 3 · riesgo alto y bajo

Se pide: dos reservas completas, una de riesgo alto sobre el papel y otra de bajo, y sus probabilidades.

Idea: copiar un caso válido y cambiar solo lo necesario: **{**base, "campo": valor}**.

Ejemplo propio: **{**prestamo_base, "dias": 42}**.

1 · Modelo y metadatos
Guarda tu mejor modelo

→

2 · logica.py
Campos, validaciones y aviso propios

→

3 · app.py
Lánzalo con **streamlit run**

→

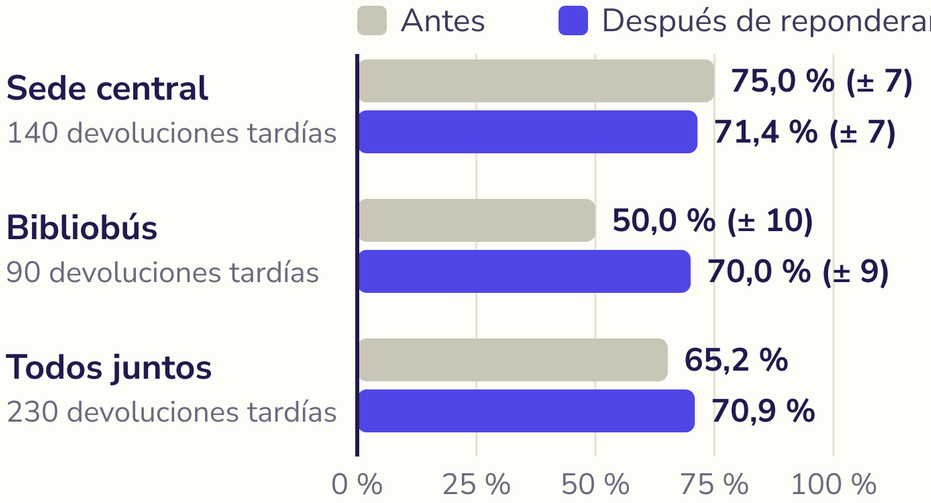
4 · Tres pruebas
De sentido común, con **pytest**

→

5 · Vídeo de 90 s
Plan B de la demo

Se autocorrigen con **assert** y llevan una «Solución de referencia» plegada. **Mini-reto:** adapta la plantilla a tu proyecto.

La media puede esconder un sesgo



Exhaustividad (recall): de las devoluciones tardías, ¿cuántas detecta el modelo? Entre paréntesis, la duda aproximada del 95 % en puntos.

Mirar por subgrupo

Con todos juntos, el modelo detecta el 65,2 % de las tardías. Pero en el bibliobús solo el 50,0 %.

Con solo 90 casos, hay una duda de unos ± 10 puntos; aun así, la brecha de 25 puntos es mayor.

Medir lo que cuesta

Más peso al bibliobús al entrenar (**reponderar**) lo sube a 70,0 %; la sede central baja 3,6 puntos.

A cambio, hay más falsas alarmas: precisión del bibliobús, de 64,3 % a 58,3 %.

Aplicar una mitigación no basta: medid antes y después, por subgrupo.

MODEL CARD Y MEMORIA (RETO) · 85 MIN

La model card: la etiqueta del modelo

1 · Detalles del modelo
Nombre, versión, librería con su versión y semilla

2 · Uso previsto
Para qué, para quién, qué decisión sigue siendo humana y qué usos NO se permiten

3 · Datos
Fuente, licencia, tamaño, cómo se partió y sesgos de muestreo

4 · Factores relevantes
Grupos o condiciones que cambian el rendimiento

5 · Métricas
Baseline frente al modelo, con su incertidumbre

6 · Rendimiento por subgrupos
Una tabla: n, métrica y diferencia con el global

7 · Análisis ético y de riesgos
AI Act y RGPD, mitigaciones y supervisión humana

8 · Limitaciones y recomendaciones
Cuándo no fiarse y qué haría falta antes de un despliegue real

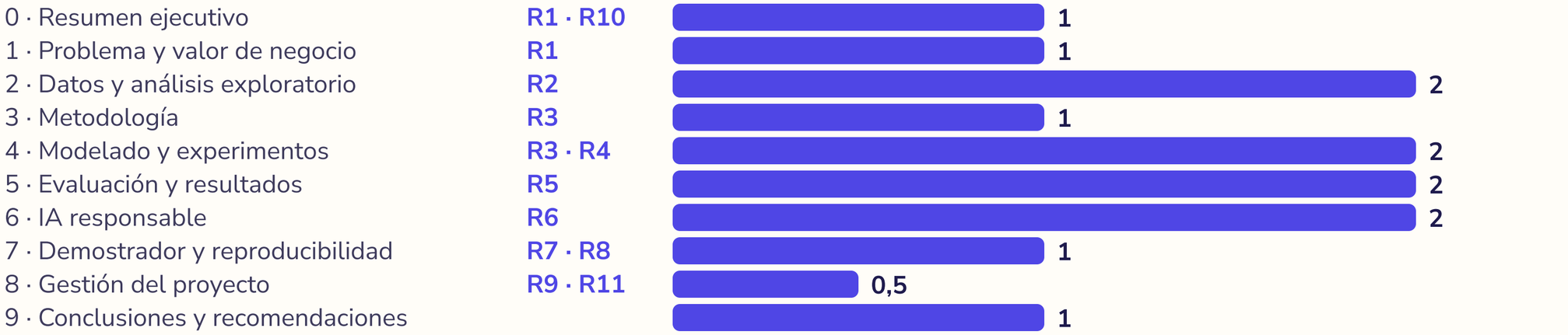
9 · Contacto y mantenimiento
Quién responde y cada cuánto se reevalúa

Máximo **2 páginas** en **docs/model_card.md**. Debe **coincidir con la memoria y con el aviso del demostrador**.

Ejemplo inventado (biblioteca), apartado 2: «Uso fuera de alcance: multar o vetar a una persona concreta.»

MODEL CARD Y MEMORIA (RETO) · ESQUELETO Y MAPA DE REQUISITOS

La memoria técnica: 10 a 15 páginas



Total: 13,5 páginas (el límite es de 10 a 15; los anexos no cuentan)

Se escribe contando lo que ya existe

Cada sección sale de algo hecho: la ficha del Sprint 14, el registro del 15 y la evaluación del 16. **El resumen, al final.**

Cada requisito, en su sección

Los R1 a R11 de la columna azul. R8 también en el README, R9 en el anexo F (tablero y retros) y R11 se demuestra en el Sprint 17.

LA IDEA · REPRODUCIBILIDAD Y CONGELACIÓN

Reproducible y congelado a las 14:30

Lista de reproducibilidad del demostrador (cuaderno, sección 7)

- ✓ **requirements.txt** con **versiones fijas** (**pip freeze**, recortado a lo que usas)
- ✓ El **modelo se crea** con un script o cuaderno y el README dice cómo
- ✓ El README dice **qué comandos ejecutar** y qué se verá
- ✓ La app **no se rompe** con entradas erróneas y avisa de la incertidumbre
- ✓ Hay **pruebas** (**pytest -q**) y un **vídeo de reserva** de 90 s
- ✓ **Ni claves ni datos personales** en el repositorio

14:30

10/12 · congelación

Último push. Antes, la etiqueta **v1.0-entrega**.

Después no se cambia ni código ni memoria (sí la presentación y el vídeo).

Prueba en limpio

Otra persona, que **no escribió el README**, lo sigue en un Colab nuevo (Code → Download ZIP) y anota lo que falla.

Push = subir los cambios al repositorio · **etiqueta (tag)** = marca de Git que señala la versión entregada · **congelación** = desde esa hora no se cambia nada. Se pasan los seis bloques de **checklist_congelacion.md**.

HOY, VIERNES 4/12

Que otra persona pueda ejecutarlo

Entregar

Con solo el README, **otra persona** lo ejecuta: versiones fijas y prueba en limpio.

Demostrar

El demostrador **no reentrena**; la lógica va aparte, los errores se explican y cada respuesta avisa de la incertidumbre.

Documentar

Sesgos **medidos antes y después**, model card de 2 páginas y memoria de 10 a 15 con los R1 a R11.

Al acabar el día: demostrador básico y esqueleto de la memoria · 9/12, casi listo · 10/12 a las 14:30, congelación.